jb, had, jvo

Lab 1: Primeiras classes e objetos

Este lab deve ser feito individualmente.

Submeta o seu código ao mooshak http://deei-mooshak.ualg.pt/~jvo/ usando o seu login da ualg (sem @ualg.pt). Ex: a123456

Uma submissão permanecerá *pending* até que seja solicitada a sua validação ao professor durante a aula prática. Só as submissões *final* serão consideradas para avaliação.

A submissão deverá ser feita até

20 de fevereiro 2026

A validação poderá ser feita posteriormente, se necessário, até 2 de março de 2026

Objetivos

1. Inscrição no Mooshak
2. Atualização do JDK e IntelliJ, se necessário
3. Desenvolvimento e validação via Mooshak de programas simples usando classes e objetos
4. Obtenção do relatório de erros do Mooshak

Inscrição no Mooshak

1. Aceda ao mooshak através do link da tutória ou http://deei-mooshak.ualg.pt/~jvo/

1.1. Para obter User/Password clique em Register [for On-line Contest]

1.1.1. Em Contest, selecione POO 2025/26

1.1.2. Em Group, selecione o seu turno prático

1.1.3. Use o seu login da ualg (sem @ualg.pt). Exemplo: o estudante como o número 123456 em Name escreve a123456 e em Email: a123456@ualg.pt

1.2. Depois de se autenticar com o seu login pessoal, pode escolher, entre outras coisas, o seguinte:

1.2.1. Problema

1.2.2. Visualizar (View) – lista o enunciado do problema selecionado

1.2.3. Perguntar (Ask) – coloque aqui as suas questões sobre o problema selecionado

1.2.4. Escolher ficheiro … Submeter (Submit) – permite fazer o upload do ficheiro com o código java ou em formato zip; mais detalhes serão dados abaixo.

1.2.5. Depois de devidamente testado, faça o upload do seu ficheiro .java, .gz, ou .zip submetendo-o.

1.2.6. Aguarde pela resposta do Mooshak e veja no Help o seu significado, se necessário.

Submissão ao Mooshak
Para submissão ao Mooshak temos duas hipóteses: ou submetemos um único ficheiro .java ou um ficheiro compactado .zip ou .gz. Estas duas hipóteses são descritas abaixo.

Hipótese ficheiro .java: o programa completo é *integralmente* incluído num único ficheiro de extensão **.java**; é esse ficheiro que é submetido ao mooshak.

O conteúdo possível desse ficheiro é:

0. opcionalmente, uma diretiva **package**;
1. zero ou mais diretivas **import**;
2. uma ou mais definições de classes, sendo que uma e só uma será pública e terá o mesmo nome do ficheiro.

Hipótese ficheiro compactado: Compacte o diretório raiz do código fonte, e todos os sub-diretorios, num arquivo .zip ou .gz; é esse ficheiro que é submetido ao Mooshak.

Por defeito, no IDE Eclipse, o diretório raiz está no Workspace, num subdirectório com o nome do projeto:

…/<Workspace>/<Nome projeto>/src

Nota 1: Tem de ser compactado o diretório *src*; não apenas os ficheiros que estão lá guardados.

src.zip
ou
src.tar.gz

Nota 2: O Mooshak vai procurar nos ficheiros uma função com o nome **main()**, para iniciar o Programa. Se houver mais do que uma, mesmo que estejam comentadas, o Mooshak vai executar a primeira que encontrar.
Para evitar que execute a função **main()** errada, tem de se garantir que só existe uma.
Alternativamente, existindo mais do que uma, pode-se indicar o caminho para a função **main()** a executar pelo Mooshak no ficheiro **mainpath.txt** a incluir logo na raiz da na pasta **src**.
Veja-se o exemplo abaixo, onde temos a seguinte hierarquia de pastas e ficheiros:

```
src
    mainpath.txt
    apps
        main1.java
        main2.java
    lib
        <ficheiros com outras classes>
        (…)
```

Temos duas funções **main()**, uma no ficheiro **main1.java** e outra em **main2.java**, ambas na

pasta **src/apps**.

Se quisermos executar a que se encontra no ficheiro **main2.java** temos de definir no ficheiro **mainpath.txt** o caminho sem incluir a pasta raiz **src** e sem a extensão do ficheiro, como no exemplo abaixo:

apps/main2

De notar que se o ficheiro indicado não existir será reportado um **Runtime Error** pelo Mooshak.

Avisos do Compilador

Alguns compiladores geram avisos (*warnings*) quando encontram caracteres que não conseguem identificar, ***mesmo nos comentários***. Isto sucede com caracteres acentuados como os portugueses (por vezes pode ser apenas uma questão de *codepage* diferente entre o sistema onde foi escrito o código e o sistema onde é compilado).

O Mooshak pode estar configurado para abortar a compilação não só quando o compilador encontra erros no código mas também quando o compilador gera avisos, sendo indicado **"Compilation error"**.

Por isso recomenda-se que se evite a utilização no código fonte de caracteres acentuados mesmo nos comentários (no caso geral caracteres cujo código ASCII é superior a 127).

Atualização do jdk e Eclipse, se necessário

1. Verifique a versão do jdk abrindo uma consola
   1.1. **Windows:** A partir do menu **Start -> Run**, escreva **cmd** e pressione **<enter>** de modo a abrir a linha de comandos.
   Linux: Numa consola em modo de texto ou abrindo uma consola virtual ou *xterm* no ambiente gráfico.
   MacOS: Abra um Terminal

   1.2. Na linha de comandos escreva:

```
javac -version
```

Caso necessário, atualize (ou instale) o JDK. O ficheiro de instalação pode ser obtido no site da Oracle, em:

https://www.oracle.com/java/technologies/downloads/

Selecione Java Development Kit 25 e faça o download correspondente ao seu sistema operativo. Note que o Mooshak irá validar o código submetido usando a JDK 25. Em Windows, após efetuado o download execute o ficheiro de instalação. Em Linux instale na linha de comandos com o gestor de pacote:

```
sudo dpkg -i jdk-25_linux-x64_bin.deb
```

Em sistemas Linux poderá também estar disponível através do gestor de pacotes de instalação, o instalador do JDK, embora possa ser uma versão não tão atualizada como a obtida no site da Oracle. Em sistemas baseados no Debian (Debian, ubuntu, etc.) a instalação do JDK livre pode ser efetuada na linha de comandos com:

```
apt-get install openjdk-25-jdk
```

Verificando a versão do JDK instalada, deve agora reportar a 25:

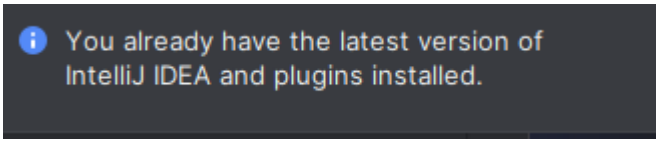
```
java  -version
javac -version
```

2. Para instalar o IntelliJ no seu computador pode usar o gestor de pacotes do sistema operativo, ou visite:

https://www.jetbrains.com/idea/

descarregue o instalador, descompacte-o e execute-o.

3. Para atualizar o IntelliJ, execute-o. Depois, vá a Help->Check for updates… Se já estiver atualizado, deve receber a seguinte mensagem:



Desenvolvimento e validação via Mooshak

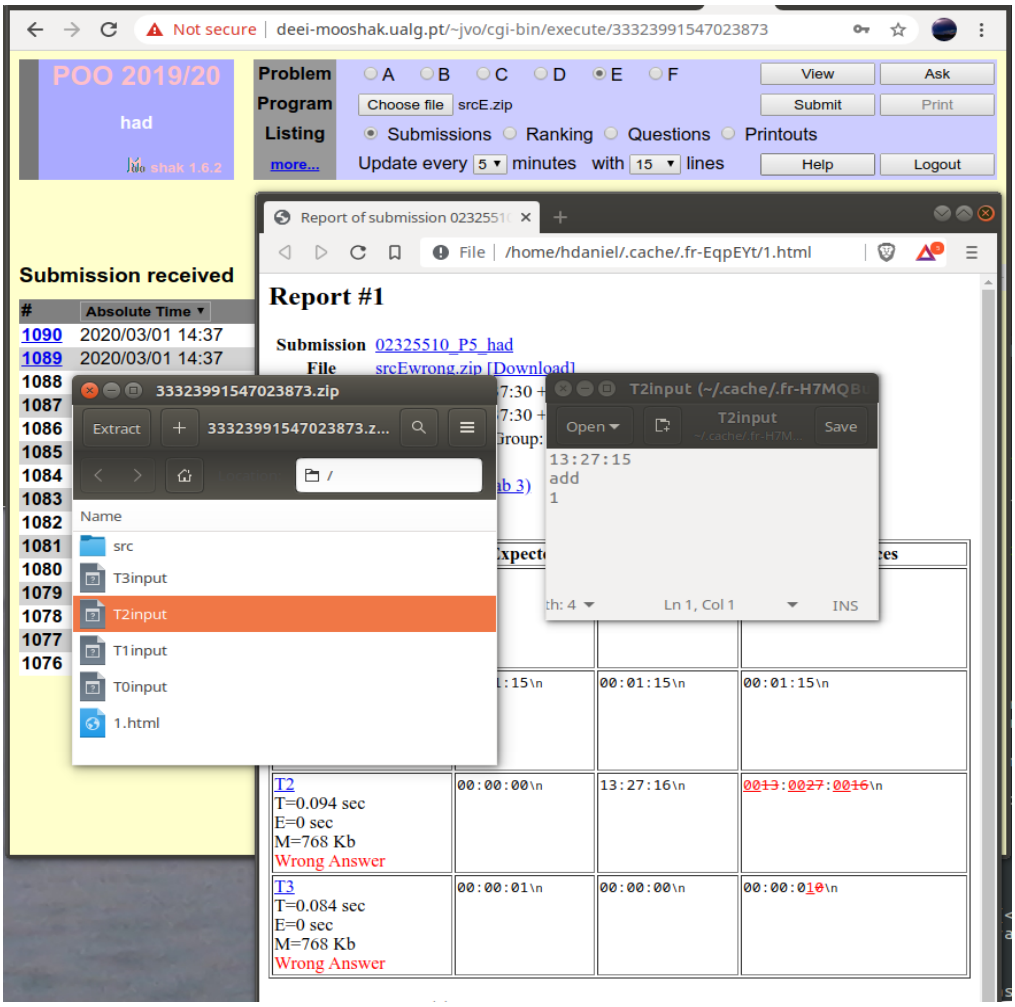
Depois de desenvolvido um programa, compacte a pasta com os ficheiros onde estão definidas as classes no código fonte com zip e submeta o zip ao Mooshak.

Veja o relatório de erros do Mooshak como descrito abaixo.

Quando tiver *Accepted* e o seu código estiver refinado, valide-o com o professor numa aula prática.

Visualização do relatório de erros do Mooshak

1. Após submissão, clique no número da submissão (o 1090, 1089,... a azul na figura abaixo) ou com o botão direito escolha **gravar como**.
2. É descarregado um ficheiro **zip**, com a submetida pasta **src**, o código, e um ficheiro **1.html**
3. Abra o ficheiro html que contém o relatório da submissão, como se pode ver na figura abaixo.
4. São também incluídos alguns ficheiros com nome **XXinput**. Cada um deles refere-se aos dados de input para cada caso de teste. No exemplo o ficheiro T2Input tem os dados de entrada para o caso de teste T2 apresentado no relatório 1.html.



Desenvolvimento de programas com as primeiras classes e objetos

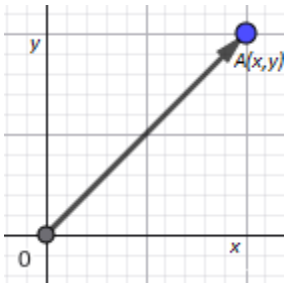
Os programas resultantes das respostas aos exercícios seguintes devem ser submetidos ao Mooshak, no problema respetivo.

Consulte agora os slides da Teóricas 1 e 2 antes de responder ao problema seguinte.

Problema A: Módulo de um vetor posição

Desenvolva um programa que receba um ponto com coordenadas cartesianas **A(x,y)** e calcule o módulo do vetor posição, isto é, um vetor definido por esse ponto e aplicado na origem **(0,0)**. O módulo é dado pela distância do ponto A que define o vetor à origem:

$$\|A\| = \sqrt{(x-0)^2 + (y-0)^2}$$



Entrada
A entrada é composta por uma linha que contém dois números reais x e y, representando as coordenadas do ponto A.

Saída
A saída tem uma única linha, que contém o módulo do vetor definido por A e aplicado na origem, arredondado a 2 casas decimais. Para imprimir um *double v* com duas casas decimais, pode-se usar:

```
System.out.println(String.format("%.2f", v));
```

Exemplo de Entrada
2 2

Exemplo de Saída
2.83

Restrições complementares
O programa deve estar estruturado em duas classes. Cada classe deve estar implementada no seu próprio ficheiro.
A classe **Vetor** que implementa o cálculo do módulo como indicado acima, cria uma nova instância através do construtor, todas as suas variáveis são privadas e não tem assessores de escrita.
A classe **Cliente** apenas lê os dados de entrada, processa-os utilizando os métodos da classe **Vetor** e escreve o resultado na consola.

Problema B: Produto interno de 2 vetores posição

Desenvolva agora um programa que, dados dois vetores posição u(x0,y0) e v(x1,y1), cada um com módulo superior a zero, calcule o seu **produto interno**, de acordo com:

$$u.v = x0.x1 + y0.y1$$

Restrições complementares
Tal como antes, o programa deverá estar estruturado nas classes **Vetor** e **Cliente**. O cálculo do produto interno deve ser efetuado apenas pela classe **Vetor**.
A classe **Vetor** também deverá verificar, cada vez que um novo objeto é criado, se o seu módulo é superior a 0. Caso esta condição seja violada, o programa termina com uma mensagem de erro, conforme descrito abaixo.
A classe **Cliente** apenas lê os dados de entrada, processa-os utilizando os métodos da classe **Vetor** e escreve o resultado na consola.

Entrada
A entrada é composta por duas linhas.
A primeira linha contém dois números reais representando as coordenadas do ponto *A* que define o primeiro vetor posição. A segunda linha é em tudo idêntica à primeira, representando as coordenadas do ponto *B* que define o segundo vetor posição.

Saída
A saída tem uma única linha, que contém o resultado do produto interno dos 2 vetores, arredondado a 2 casas decimais, se o módulo de cada um dos vetores for superior a 0; caso contrário, termina com mensagem "iv" que significa *invariant violated.*

Exemplo de Entrada 1
2.0 1.0
6.2 3.5

Exemplo de Saída 1
15.90

Exemplo de Entrada 2
0.0 0.0
20.0 3.5

Exemplo de Saída 2
iv

Problema C: Cossine similarity

Desenvolva agora um programa que, dados dois vetores posição u(x0,y0) e v(x1,y1), cada um com módulo superior zero, calcule a sua ***cossine similarity***, de acordo com:

$$S_c(u,v) = \frac{u.v}{\|u\|.\|v\|}$$

Restrições complementares
Tal como antes, o programa deverá estar estruturado nas classes **Vetor** e **Cliente**. O cálculo da ***cossine similarity*** deve ser efetuado apenas pela classe **Vetor**.
A classe **Vetor** também deverá verificar, cada vez que um novo objeto é criado, se o seu módulo é superior a 0. Caso esta condição seja violada, o programa termina com uma mensagem de erro, conforme descrito abaixo.
A classe **Cliente** apenas lê os dados de entrada, processa-os utilizando os métodos da classe **Vetor** e escreve o resultado na consola.

Entrada
A entrada é composta por duas linhas.
A primeira linha contém dois números reais representando as coordenadas do ponto *A* que define o primeiro vetor posição. A segunda linha é em tudo idêntica à primeira, representando as coordenadas do ponto *B* que define o segundo vetor posição.

Saída
A saída tem uma única linha, que contém o resultado da ***cossine similarity*** dos 2 vetores, arredondado a 2 casas decimais, se o módulo de cada um dos vetores for superior a 0; caso contrário, termina com mensagem "iv" que significa *invariant violated.*

Exemplo de Entrada 1
1 3
4 5.5

Exemplo de Saída 1
0.95

Exemplo de Entrada 2
1 1
0 0

Exemplo de Saída 2
iv

Alguns aspetos práticos sobre o Mooshak
(Baseado em http://ctp.di.fct.unl.pt/~amd/cpn/2007tiup/etapa5/praticos.html)

Dados de entrada
Os dados de entrada são lidos da entrada padrão. Consistem em texto cuidadosamente formatado para ser simples de processar:

- Normalmente, nas primeiras linhas dos dados de entrada surgem alguns números inteiros que anunciam o tamanho das diversas partes do texto que se segue. Isso evita a necessidade de testar a condição de "fim-de-ficheiro", durante a leitura dos dados.
- A última linha do ficheiro está sempre devidamente terminada por uma mudança de linha.
- Espaços em branco, quando usados, são sempre considerados como separadores. Os espaços em branco nunca ocorrem em grupos. Uma linha nunca começa com um espaço em branco. Uma linha nunca acaba com um espaço em branco.

  Note que as linhas com números inteiros que ocorrem no início dos dados de entrada devem ser lidas até ao fim para evitar desalinhamentos na leitura dos dados subsequentes.

  Eis como se lê um inteiro em Java:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
/* Aviso: nunca crie mais do que um Scanner sobre o input. */
int n = sc.nextInt();
```

- Supondo que os dados se iniciam por uma linha contendo dois inteiros separados por um espaço em branco:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
int n = sc.nextInt();
int m = sc.nextInt();
```

NB: Em alternativa a

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

escreva apenas

```
Scanner sc = new Scanner(System.in);
```

Mas para isso terá que escrever no inicio do ficheiro de código:

```
import java.util.Scanner;
```

Dados de saída
Os dados de saída são escritos na saída padrão.

Todas as linhas de saída terminam com “\n”. Exemplo: Use

```
System.out.println("iv");
```

Para escrever na saída a String literal “iv”.

É necessário respeitar rigorosamente o formato exigido no enunciado.

Qualquer desacerto, mesmo ligeiro, é suficiente para que um programa seja classificado como "Presentation error".

Note que não é possível detetar visualmente certas anomalias nos dados de saída. Por exemplo: um espaço em branco no final duma linha, uma linha em branco no final dos dados, a omissão da mudança de linha na última linha dos dados. Todas estas situações são inaceitáveis e provocam um "Presentation error".